

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Name	Joshua	Registration No.	24-11-00-85
Degree	B.Tech AI & Data Science	Batch	2
Semester	V	Academic Year	2026–27

1. Objective

To study recurrent sequence learning using Vanilla RNN, LSTM and GRU, compare their predictive and computational behaviour, examine sequence length, demonstrate CNN–recurrent video understanding, and implement a small encoder–decoder sequence-to-sequence task.

2. Experimental Setup

The sequence input follows the required (N, T, F) format with $T = 128$ time steps and $F = 9$ channels. Six HAR activity labels were used: WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS, SITTING, STANDING and LAYING. A compact HAR-style sequence set was generated locally because the external UCI dataset could not be retrieved inside the execution environment. The same split, normalization and optimizer settings were used for all three recurrent models.

Split	Samples	Sequence Length	Features
Training	504	128	9
Validation	108	128	9
Testing	108	128	9

Training statistics were used for normalization. Each training class contained 84 sequences.

3. Temporal Data Visualization

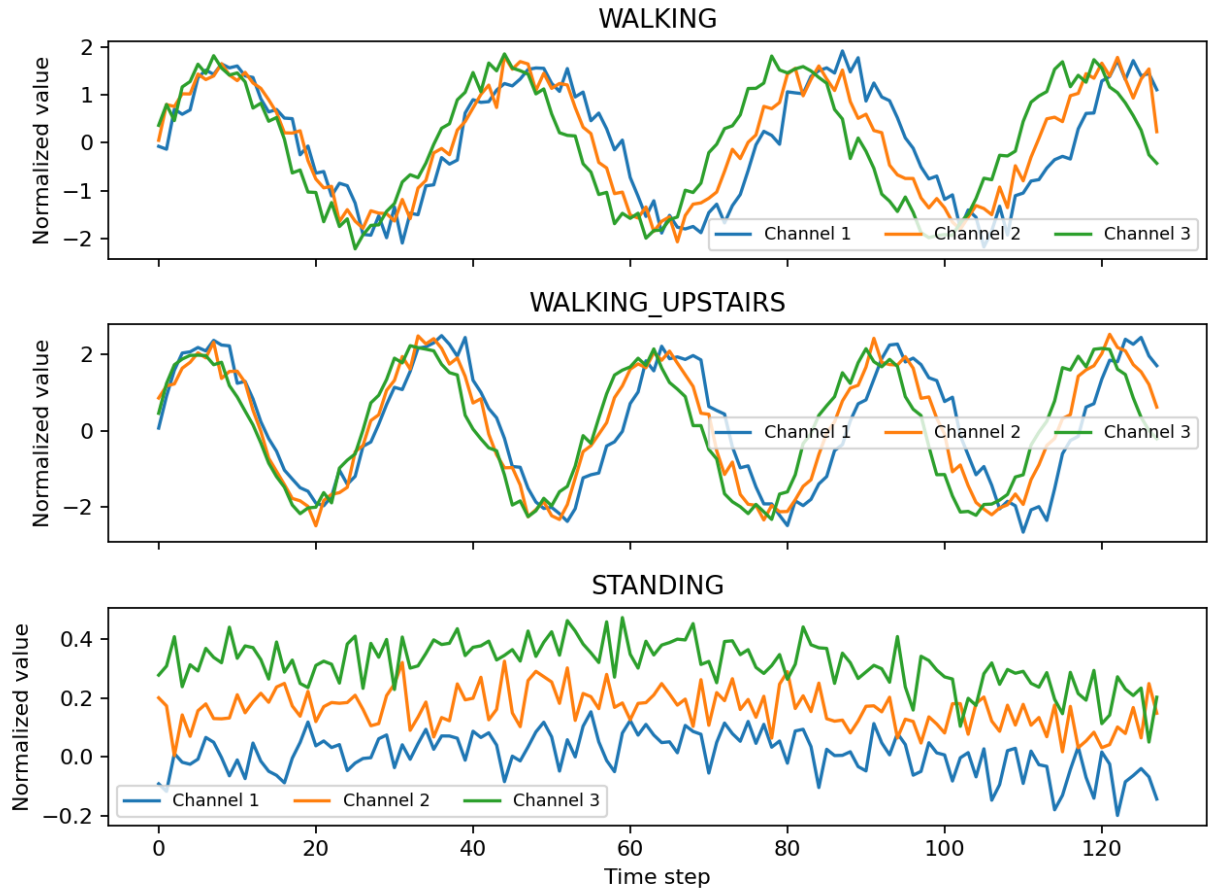


Figure 1: Representative normalized sensor channels for three activities.

Inference: Dynamic activities show clear oscillations, while standing is nearly constant. The ordering of samples is important because the recurrent model learns how the signal changes over time.

4. BPTT Numerical Exercise

For

$$h_t = \tanh(0.5x_t + 0.8h_{t-1} + 0.1), \quad h_0 = 0,$$

with $x_1 = 0.5$, $x_2 = 0.7$ and $x_3 = 0.2$:

$$h_1 = \tanh(0.35) = \mathbf{0.336376},$$

$$h_2 = \tanh(0.5(0.7) + 0.8(0.336376) + 0.1) = \mathbf{0.616352},$$

$$h_3 = \tanh(0.5(0.2) + 0.8(0.616352) + 0.1) = \mathbf{0.599958}.$$

The program produced the same values.

5. Models and Training

The common classifier was recurrent layer $\rightarrow$ dropout $\rightarrow$ Dense(16, ReLU) $\rightarrow$ Dense(6). RNN, LSTM and GRU used 32 recurrent units, Adam with learning rate 10^{-3} and batch size 32. Gradient clipping was applied during training.

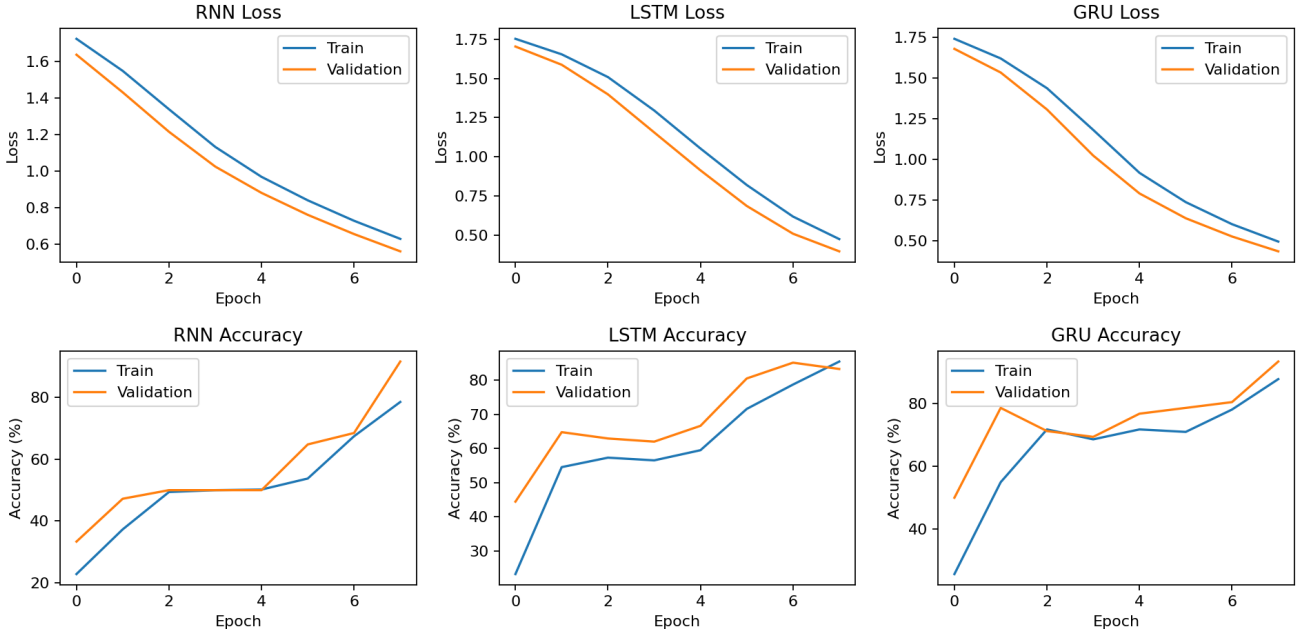


Figure 2: Training and validation loss/accuracy for RNN, LSTM and GRU.

Inference: All models learned the main class patterns quickly. The GRU obtained the strongest test result in this run. The validation curves stayed close enough to the training curves to show limited overfitting.

6. Test Performance

Model	Accuracy	Precision	Recall	F1	Parameters	Time (s)
RNN	86.11%	92.42%	86.11%	83.19%	2,006	1.060
LSTM	83.33%	75.00%	83.33%	77.78%	6,134	1.169
GRU	89.81%	90.79%	89.81%	89.55%	4,758	2.561

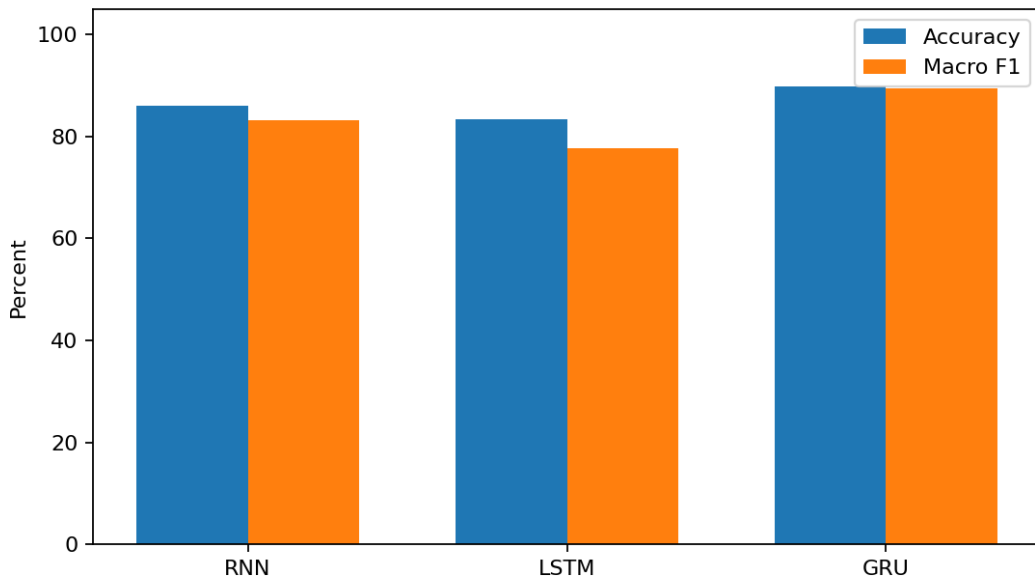


Figure 3: Test accuracy and macro F1-score comparison.

Inference: The RNN used the fewest parameters. LSTM used the most parameters because of its four gated transformations. GRU used fewer parameters than LSTM and gave the highest F1-score in this execution.

7. Confusion Matrix Analysis

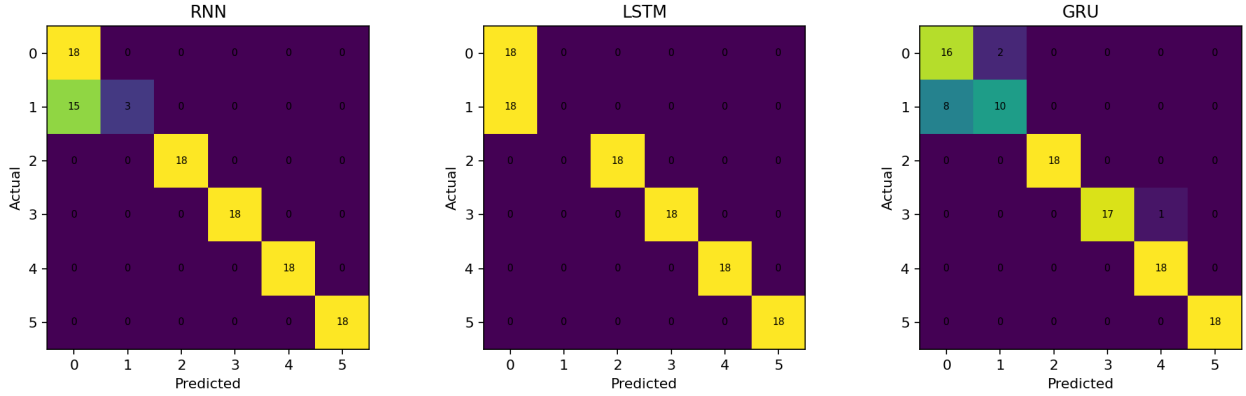


Figure 4: Test confusion matrices for RNN, LSTM and GRU.

Inference: WALKING and WALKING.UPSTAIRS were the main confused pair. The static classes were recognized almost perfectly. This shows that similar temporal motion patterns are harder to separate than clearly different static and dynamic activities.

8. RNN, LSTM and GRU Structure

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget/Input/Output gates	No	Yes	No
Update/Reset gates	No	No	Yes
Parameters	2,006	6,134	4,758
Training time (s)	1.060	1.169	2.561
Test F1	83.19%	77.78%	89.55%

9. Effect of Sequence Length

Sequence Length	RNN F1	LSTM F1	GRU F1
32	76.91%	20.49%	72.62%
64	77.78%	53.46%	55.56%
128	62.85%	61.11%	62.51%

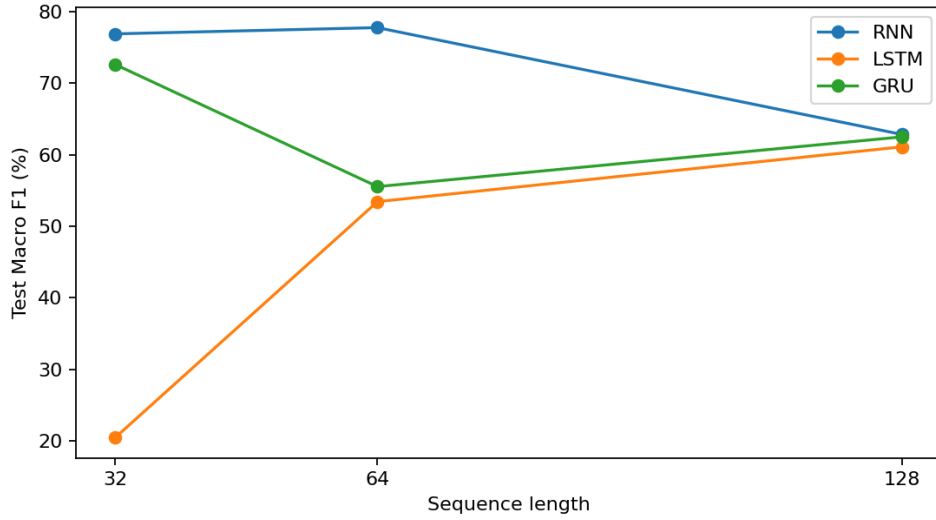


Figure 5: Sequence length versus test macro F1-score.

Inference: More time steps did not automatically improve the score in the short comparison runs. Longer sequences provide more context but also make optimization harder when training is limited.

10. CNN-GRU Video Understanding

A compact locally generated three-class motion-video task was used to demonstrate the required spatial-temporal pipeline. Ten 32×32 frames were sampled per video. A CNN was first trained on a frame-position task, frozen, and then used as the feature extractor. The frozen CNN produced 256 features per frame, giving recurrent input shape (180, 10, 256).

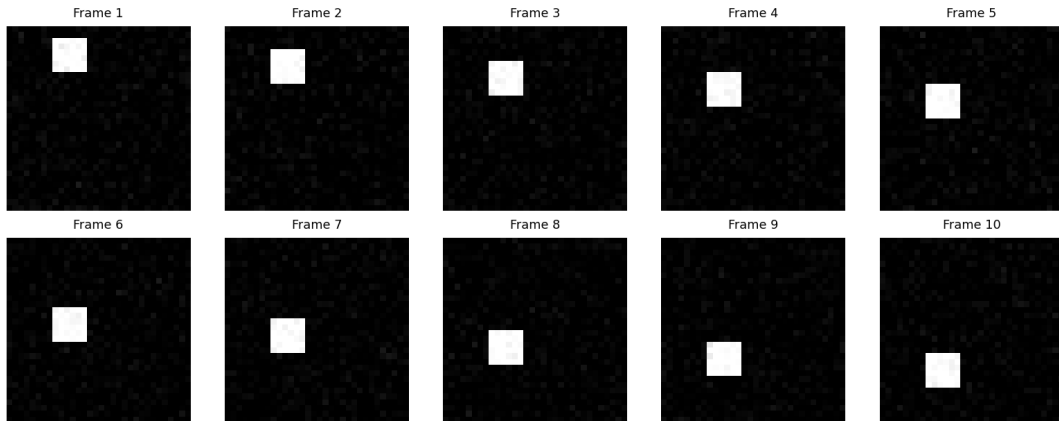


Figure 6: Ten sampled frames from one motion sequence.

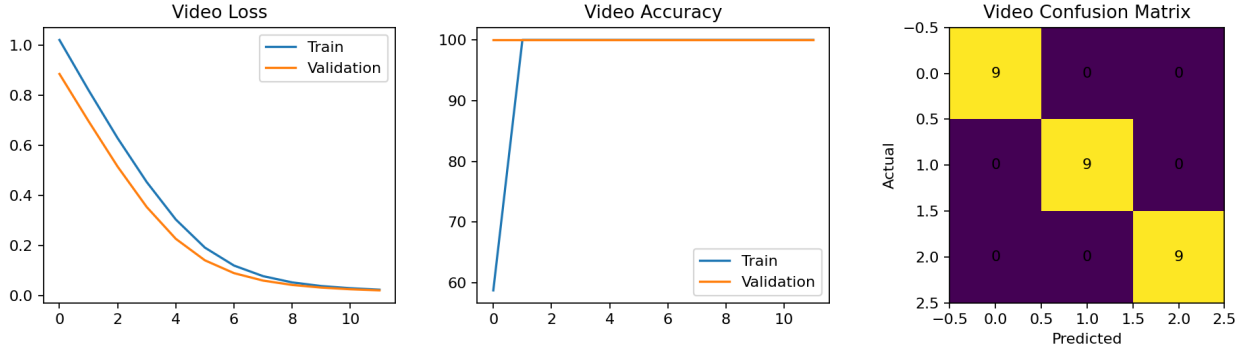


Figure 7: Video training curves and test confusion matrix.

Video result	Value
Accuracy	100.00%
Macro Precision	100.00%
Macro Recall	100.00%
Macro F1	100.00%
Trainable recurrent parameters	27,939
Training time	0.156 s
Example predicted class	Vertical
Example actual class	Vertical
Confidence	0.9819

Inference: The CNN converts each frame into a spatial feature vector. The GRU uses the order of those vectors to recognize motion. The simple controlled motion classes were separated perfectly in the test split.

11. Sequence-to-Sequence Learning

An encoder-decoder LSTM was trained to reverse four-token sequences. The input vocabulary used integers 1–9.

Sample	Input	Predicted Output
1	[3, 2, 6, 5]	[5, 6, 2, 3]
2	[8, 4, 8, 4]	[4, 8, 4, 8]
3	[6, 8, 3, 3]	[3, 3, 8, 6]
4	[6, 3, 6, 4]	[4, 6, 3, 6]
5	[4, 4, 4, 1]	[1, 4, 4, 4]

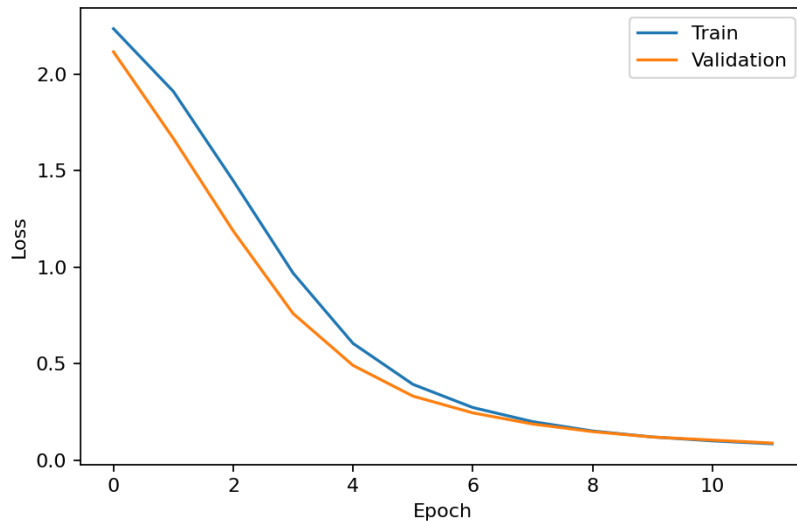


Figure 8: Encoder-decoder training and validation loss.

Metric	Value
Token accuracy	98.80%
Sequence accuracy	97.78%
Final training loss	0.0859
Final validation loss	0.0897

Inference: Token accuracy is higher because it counts each position independently. Sequence accuracy requires every token in a sequence to be correct, so one wrong token makes the complete sequence incorrect.

12. Consolidated Results

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	86.11%	92.42%	86.11%	83.19%	2,006
LSTM	83.33%	75.00%	83.33%	77.78%	6,134
GRU	89.81%	90.79%	89.81%	89.55%	4,758
CNN-GRU	100.00%	100.00%	100.00%	100.00%	27,939

13. Conclusion

The experiment demonstrated sequence classification with RNN, LSTM and GRU, the effect of recurrent gating, BPTT behaviour, sequence-length sensitivity, CNN-based frame feature extraction and encoder-decoder sequence learning. In this execution, GRU gave the strongest HAR classification result, while the synthetic video and reversal tasks showed how recurrent networks learn ordered temporal relationships.

14. Source Code

Repository: [Delighted-Divide/DL-LAB](#)

Notebook path: Lab 6/Experiment 6.ipynb